

---

# CSC 412 – Operating Systems

## Programming Assignment 03, Fall 2025

Wednesday, October 1st, 2025

---

**Due date:** Thursday, October 9th, 11:59pm.

## 1 What this Assignment is About

### 1.1 Objectives

The objectives of this assignment are for you to

- Write C code making use of 1D and 2D arrays, and exploiting pointer algebra;
- Work with a small third-party library (here, to load and write TGA images);
- Write some simple image processing applications;
- Get some practice with `fork()` and the family of `execXY()` functions
- Search through a directory in C;
- Write a small utility that you could extend later on to something actually useful..

This is an individual assignment.

### 1.2 Handout

Besides this document as a pdf file, the handouts for this assignment are the image library from Lab 04, a small C program that shows you how to use functions of the library and access directly the raster of an image object, and the `build.sh` build script from Lab 04.

## 2 Part I: TGA Image Files and the Code Handout

### 2.1 The TGA file format

The TGA (Targa file) image format is one of way-too-many image formats that you may run into. Other formats provide better compression or support for a wider range of colors, so why use this format at all? Because the format for *uncompressed* `.tga` image files is the only truly multi-platform format that is easy to read and write<sup>1</sup>. We could use a more complete library such as `freeimage` or `ImageMagick`, but, as long as your image file stores data uncompressed and does not contain any comments, the code supplied here will do the job.

---

<sup>1</sup>Yes, I know about PBM/PGM/PPM. I have been using them for the past 30+ years. They're just terrible.

## 2.2 The code supplied

The code supplied consists of:

- The “Image Library,” which is organized as follows:
  - An `include` folder containing header files that define the `RasterImage` type (in `rasterImage.h`) and image processing operations supported by the library. For example, `ImageIO.h` gives the prototype of the `readImage` function, while `ImageIO_TGA.h` gives that of the function specialized for reading images in the TGA file format. Many of the header files define the prototype of a single function, for example `flipV.h` defines the function `flipV` that performs vertical mirroring on an image.
  - A `src` folder containing source files in which these data types and functions are implemented. I decided to regroup all the image processing functions in the source file `operations.c`.
  - An `applications` folder containing source files for standalone applications that use the functions of the library.
  - An `executables` folder, where the built executables of the applications are stored.
  - A `library` folder that we won’t use for the time being, but where we may produce static or dynamic binary libraries that a program can link against.
- `main.c` is the code of a small program that shows how to use the library to read and write image files, perform some operations, and how to access the pixel data of an image.
- The `Images` folder contains a few small images in the TGA format.
- The `Output` folder is where my `main.c` program was hard-coded to produce its results, but the different executables can accept paths to other output folders.

## 2.3 About the `rasterImage.h` header file

I want to attract your attention to a few important points regarding this header file.

First, if you are familiar with javadoc style of comments, you probably recognized the characteristic `/**`. For several years, C++ programmers secretly envied their Java-programming colleagues who could produce good-looking code documentation through the javadoc system. They envied them until a brave soul came up with Doxygen, which interprets javadoc-style comments in a multitude of programming languages (C, C++, Python, Perl, etc.). Therefore, coding in C is not a good excuse to have a lousy documentation anymore, and, starting with this assignment, you will be expected to write javadoc-style comments and produce documentation in the form of html files. I will return to this point later in this document.

Second, the two `enum` types, `ImageFileType` and `ImageType`, mention types that we won’t encounter in this assignment. We will only deal with `TGA_COLOR` images that we will store as `RGBA32_RASTER`. I might as well observe here that I love using `enum` types to represent and group constants that belong together. I already alluded to that in the C code samples that I posted on Brightspace at the beginning of the semester. Enumerated types work well with `switch`

statements, are easy to expand, and have all the advantages of an integral type (for calculations and array indices) while offering some—limited—protection regarding range.

Next, the `RasterImage` type stores the 1D raster master pointer as a `void*` pointer. This is a common design decision in C (and even C++), when the type of the data could be any of multiple possible types (here, besides the 4 bytes of color information that we will be dealing with, we could have a float value or a single byte). Set the pointer to be `void*`, and cast it back to the proper type when the data must be processed. We will encounter this pointer trick several times this semester.

Finally, the `RasterImage` type contains a—commented-out—field for a 2D raster (from the name, but it is also cast to a `void*` pointer). This field is currently not used anywhere in the handout’s code, because the corresponding 2D array has not been allocated. This will be a task for you to complete when I propose you later in this assignment to “scaffold” a 2D array on top of an existing 1D array (for extra credit).

## 2.4 How to build the handout

Let’s say that I want to build the program from the root directory of the handout. The source file are easy enough to access from here, but if we try to execute

```
gcc -Wall -std=c11 Code/ImageLibrary/src/rasterImage.c\  
Code/ImageLibrary/src/imageIO\_TGA.c Code/main.c -o prog
```

we get compiler errors because the compiler cannot find the header files `rasterImage.h` and `imageIO_TGA.h`.

**Note:** Before we go any further, please note that I must list all the source files in my compiling statement. The `\` character in the above command is there to indicate to the shell that the command continues on the next line. In effect, this `\` character “eats” the end-of-line character. However, if you type this command, be careful to hit the return key right after the `\`, without adding a space character.

While I am making general comments, I might as well mention that instead of listing all the source files in the `src` directory one by one, I can say “all the file `.c` in that directory” by using the wildcard character `*`

```
gcc -Wall -std=c11 Code/ImageLibrary/src/*.c Code/main.c -o prog
```

**Back to the header problem** Right now, the compiler doesn’t find the header files. First, let’s eliminate the two obvious incorrect “solutions”

- Regroup the header files and source files in the same directory;
- Edit the source files to change the path to the header files in the `#include` statements (e.g. `#include "../include/rasterImage.h"`)

You shouldn't do that because, first, this is not your library but a part of the handout, and you have no business modifying it to create your own personal customized version of it that will be incompatible with the “mainstream” version. Second, you can easily see how such an approach would not fare well if you were dealing with a substantially larger library, with dozens of source and header files.

The proper way to handle this problem is to indicate to the compiler where to look for header files:

```
gcc -Wall -std=c11 -I Code/ImageLibrary/include\  
Code/ImageLibrary/src/*.c Code/main.c -o prog
```

Note that if we needed the compiler to look into more than one director, we could just chain the `-I <directory path>` modifiers.

## 2.5 What the program does

The program doesn't take any argument. It hard-codes the paths to a TGA input image files and three output image files. The program read the input image into an `RasterImage`, performs two simple operations on the raster data: (“kill” one of the color channels) and flip the image vertically, and writes the resulting images as new TGA files. It also creates a new `RasterImage` from scratch and fills it with some color values. In other words, it demonstrates all the elementary operations that you will need to combine to complete this assignment. Make sure that you understand well all parts of the code. Post questions on the forums.

## 2.6 Color images and rasters

Pretty much all libraries dealing with image and video data<sup>2</sup> manipulate image data under the form of a 1D “raster,” as shown in Figure 1.

So, if our image has  $m$  rows of  $n$  columns and each pixel is stored on  $k$  bytes (for a total of  $n \times k$  bytes per row and  $m \times n \times k$  bytes for the entire image), then the pixel at row  $i$  and column  $j$  in the image can be addressed at index  $i \times n \times k + j \times k$  of the 1D raster.

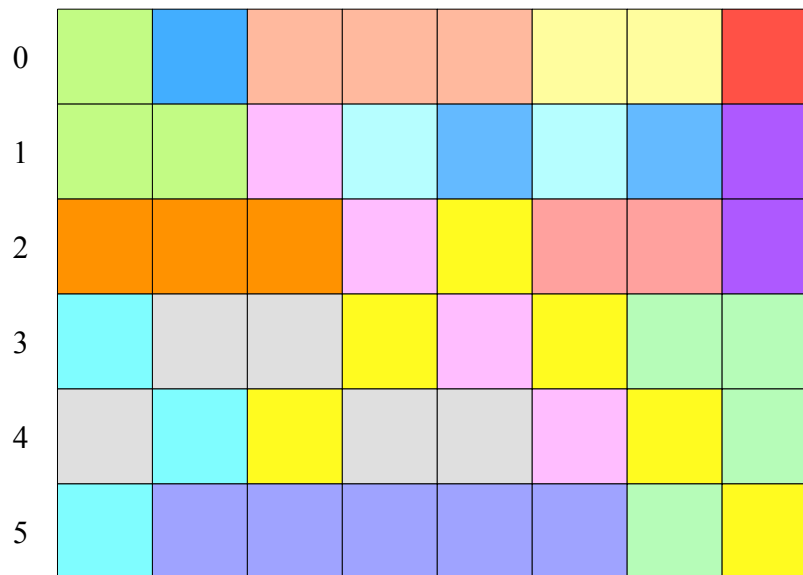
In the uncompressed TGA file format, an image file stores each pixel as 3 unsigned bytes (range 0 to 255) encoding the red, green and blue color channels. When we load an image in memory, however, it is more convenient to store each pixel on an even 4 bytes. Besides allowing image processing and other graphic applications to use this additional byte to encode *transparency* (or the “ $\alpha$  channel,” as it is called), this also means that a pixel occupies the same space as an `int`, so that we can view our 1D raster either as an `int*` pointing to a pixel or as an `unsigned char*` pointing to a color channel of a pixel.

Note that different applications may store the bytes of the color channel in different orders. The most common orders are `argb` (alpha, red, green, blue) and `rgba`. Throughout this assignment, and future assignments dealing with images, we will use exclusively the `rgba` format.

---

<sup>2</sup>An important exception is Apple's old QuickTime library and the `libquicktime` open-source replication of the API for a tiny bit of QuickTime. I mention this here because we may have an encounter with `libquicktime` before the end of the semester.

## The image as we see it



## The raster that stores the image

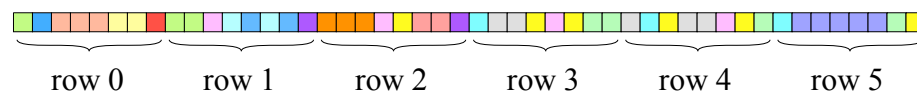


Figure 1: An image stored as a 1D raster.

What this means is that if I want to access the color information of the pixel at row  $i$  and column  $j$  of the same  $m \times n$  image, then I could get a pointer to this pixel's color channels, each seen as an unsigned char by writing

```
unsigned char* rgba = (unsigned char*)raster + 4*(i*n + j);
```

Now, I can view this pointer as an array and access directly my color components:

- `rgba[0]`: red channel,
- `rgba[1]`: green channel,
- `rgba[2]`: blue channel,
- `rgba[3]`: alpha channel.

Alternatively, I could see my pixel as a 4-byte int that I access through an `int*` pointer by writing

```
int pixel = *((int*)raster + i*n + j);
```

Now I can access my color channels by extracting the different bytes of the `pixel` variables. There is just a small problem: the byte order. Different CPU architectures have a different way to encode `int`, `float`, etc. Or rather, the encoding is the same, but the byte order is different. Intel processors (and their AMD clones) are “small-endian,” which means that their least significant byte is stored first, and their most significant byte is stored last. On the other hand, Apple’s current line of Mx (M1, M2, M3, M4) processors are big-endian and store the most significant bit first and the least significant bit last.

## 2.7 Two pieces of warning

At this point, I would like to give you a piece of warning that applies to all types of “grid-based” problems (of which image processing problems are a kind): Do not ever, EVER, use `x` and `y` indices to refer to column and row indices. Use `i`, `j`, `k`, `l`, `row`, `col`, but **not** `x` and `y`. The reason for this is simple: in 2D arrays and grids, the first index is a row index, that is, a “y” index. So, if you were using `x` and `y`, you should always write `myGrid[y][x]`. However, because we are so used to the `xy` order, I can guarantee you that once and a while you will write `myGrid[x][y]`, and get this way a pretty hard-to-find bug.

Personally, in my code, I always use

- `i`, `k`, `row` for row indices,
- `j`, `l`, `col` for column indices.

No exception.

A second piece of warning, when working on grid-based problems, is that you should try to work with non-square grids, some longer than wide, some the other way around<sup>3</sup>. The reason for this is that this makes it easier to spot when you inadvertently switch row and column coordinates in your code.

## 3 What to Do, Part II: Image Operators

You are going to implement several small programs that all perform a simple operation on an image. These programs will all use the handout code provided to read and write images.

### 3.1 How development will proceed

For this assignment, you are going to wear alternatively two kinds of hats:

- While implementing, say, image rotation, you are adding functionality to the image library, and so you are wearing your “library developer” hat. Your *user* is another programmer, who is going to call your cool new functions for their IP development.

---

<sup>3</sup>Of course, for some problems this is not an option: If you write a chess program, then the grid is  $8 \times 8$ , whether you like it or not.

- While developing a small program that performs image rotation, you are going to call the brand new “rotate an image” function of the image library, and you are wearing your “application developer” hat: Your user is someone who is running your application with an image to process.

You are going to regroup all the image processing *functions* that you are going to develop into a single source file `operations.c` and companion header file `operations.h`, to be added to the image library. And then we are going to create a new directory named `Applications` alongside `ImageLibrary`, at the root level of your project directory. You are going to move the original `main.c` source file from the handout to the `applications` directory, and create there a new “main” source file for all the applications that you need to implement: `dimensions`, `rotate`, `crop`, `split`.

## 3.2 Extra credit 1: Implement the 2D raster (5 points)

[Yes, we start right away with EC. You don’t *have to* complete this EC to do any part of the assignment, but having access to the 2D raster makes some the tasks easier (or at least make the code easier to debug), in my opinion.]

Modify the code of `newImage` in `RasterImage.c` and `readTGA` in `ImageIO-TGA.c` to add the implementation of the 2D raster. Remember that this 2D raster does not constitute a duplication of the storage. It is just a *scaffolding* on top of the 1D raster giving you a more convenient way to access the data of the 1D raster. Verify that your implementation is correct by using the 2D raster rather than the 1D raster to “kill” that color channel in a modified version of `main.c` named `main2D.c`.

Mention in your report that you have implemented this EC, because its implementation won’t be immediately visible to us and we may miss it while grading.

## 3.3 Report dimensions

### 3.3.1 What it does

This program named `dimensions` takes as argument the path to a TGA image file and reports the dimensions of the image. This program may be called with different optional modifiers that affect the output of the program:

- `-h` Output image height only;
- `-w` Output image width only;
- `-v` verbose output.

The modifier `-v` may be combined with `-h` or with `-w`, in either order, but the modifiers `-h` and `-w` are mutually exclusive.

Assuming that the program is executed with the image file `clown.tga` provided with the handout (that image has a width of 640 and a height of 640), possible outputs would be:

- `dimensions ../Images/clown.tga`  
→ 320 200

- `dimensions -h ../Images/clown.tga` → 200
- `dimensions -w ../Images/clown.tga`  
→ 320
- `dimensions -v ../Images/clown.tga`  
→ width: 320, height: 200
- `dimensions -vh ../Images/clown.tga`  
→ height: 200
- `dimensions -wv ../Images/clown.tga`  
→ width: 320

### 3.3.2 Error handling

Your program should report if the file is not of the proper type (wrong file extension), if the image is not found (invalid path) or cannot be read, or if invalid optional modifiers are passed to the program. For example:

```
./dimensions -v someFile.txt
The argument is not a tga file:  someFile.txt
./dimensions /notThere.tga
File not found:  /notThere.tga
./dimensions -h -v ./image.tga
Multiple modifier chains not allowed.
./dimensions ./badFile.tga
File cannot be read ./badFile.tga
./dimensions -z ./image.tga
Invalid modifier:  -z
./dimensions -vzh ./image.tga
Invalid modifier:  -z
./dimensions -vwh ./image.tga
Modifiers -w and -h cannot be used together
```

### 3.3.3 How to implement it

The source file for this program should be named `dimensions.c` and be located in the `Applications` directory. This program will call a function implemented in the `operations.c` source file that should get an image and output preferences as arguments and provide the dimensions of the image. Let's first agree on what the function *should not* be:

```
void printDimensions(char* filePath) {
    RasterImage img = readImage(filePath)
    get image dimensions
    printf the image dimensions
}
```

What is wrong about this function is that it is tied to a file input and a terminal output. But you may want to work with image object not coming from a file (for example, results of some IP operation, or images captured from a webcam. Second, why only work with terminal output? What if you wanted these dimensions to go to a file output? Would you need to write a second function? And then a third one if the dimensions were supposed to be displayed in a GUI? It is generally a bad idea to mix input/output stuff and computation stuff in the same function: It is preferable and more versatile to have functions doing only computations and functions dedicated to input or output.

We address the first problem by sending to our function a *pointer* to a `RasterImage` object. Why a pointer to a `RasterImage` rather than just a `RasterImage`? Because, this way, we are not making and passing a *copy* of the object. We address the second problem by having our function return a C string. C strings are the only arrays in C that store their size (here, implicitly, thanks to the `0` terminating character. So, our function should be declared in its header file as

```
char* getDimensions(RasterImage* image);
```

or some other name if you don't like that one.

**What about the -v, -w, -h modifiers** Oh yes, your function should take at least one more argument specifying the format of the string produced. I will leave this for you to ponder and design.

## 3.4 Split color channels

The source file for this program should be named `split.c` and be located in the `Applications` directory. The program takes as arguments the path to a valid TGA image file and the path to an output directory. It outputs in the output directory the three image files obtained by separating the red, green, and blue channels of the input image. If the name of the input image was `myImage.tga`, then the output images should be named `myImage [r].tga`, `myImage [g].tga`, and `myImage [b].tga`.

### 3.4.1 Error handling

Your program should report if the file is not of the proper type (wrong file extension) or if the image is not found (invalid path). For example:

```
./split someFile.txt
```

```
The argument is not a tga file:  someFile.txt
```

```
./split /notThere.tga
```

```
File not found:  /notThere.tga
```

## 3.5 One More Image Operation

### 3.5.1 What it does

This program built as `compare`, should take as arguments the paths to two image files in the TGA format, and return the value 1 if the image stored in the first file is identical to the one stored in the second one, and returns 0 otherwise.

I fully understand that this is an abuse of the return value of a C program, but this is what we are going to do to keep the problem simple. Just to explain what I just said, the value that you return

(or should return) at the end of your `main` function or when you make a call to `exit(<value>)` somewhere in the program does have a meaning. The convention is that

- a return value of zero indicates normal termination, “all OK, no error occurred during execution.”
- non-zero return values are error codes. Some error codes are used by the system:
  - On Windows, a negative error value is a *system error* code, reported directly by the OS;
  - On a \*NIX system, the OS uses some specific error codes: 1 (covers a bunch of problems, including division by 0), 2 (permission denied), 126 (permission related to the mode, fixed by `chmod`, as we have already seen), 127, 128, 130, and 255.
  - And then there are application-specific error codes, the ones that you write in your code.

Now you can see why I say that the `compare` application that I ask you to implement somewhat abuses the above convention. As I said, this is to keep things simple for this assignment. We will learn this semester better ways to report back to the parent process.

### 3.5.2 Implementation details

We are going to do just as we did for `rotate`, `crop`, etc.: The function prototype will get added to `operations.h`, the code of the function to `operations.c`, and the source code of our `compare` main program will go in the `Applications` folder.

### 3.5.3 Extra credit (3 points)

Add a “fuzzy” version of `compare` named `compareFz` that returns 1 if the images only differ by 5% or less, and 0 otherwise.

### 3.6 Extra credit (6 points)

Add a “smart” version of `compare` named `compareSm` that still recognizes images that have been flipped vertically or horizontally.

## 4 Part III: Dispatched Search for Image Matches

### 4.1 Organization

The source file of your program should be named `matchSearch.c` and be located in the `Programs` folder (not the `applications` folder of the Image Library. The executable will be built `asmatchSearch`. As usual, the “public” functions that you implement should have a prototype in the `include` folder, while “private functions” that you may need to implement, but that developer users need not even be aware of, should have their prototype directly in `operations.c`.

## 4.2 What it does

Your program will be launched from the command line with three arguments:

1. the path to an image file in the uncompressed TGA file format;
2. the path to a “search” folder containing images (possibly among other files) in the same TGA file format;
3. the path to an output folder.

For example:

```
./matchSearch Images/clown.tga ../Data ./Output/
```

Your program will proceed to look, for any image file in the search folder, for possible matches (images that “compare” as being the same as the input image) in the search folder. Your program will write in the output folder a text file giving the list of matches found. The text file will be named after the image file. In the execution example above, the file written in the folder `./Output` would be named `clown.txt`.

The file would contain, on separate lines,

1. The path to the input image,
2. The path to the search folder,
3. The number of matches found,
4. A list of the *names* of the files that were a match.

For example

```
Images/clown.tga
../Data
3 image1.tga image23.tga stuff.tga
```

or, in the case where no match was found

```
Images/clown.tga
../Data
0
```

Your program does not need to check if the search and output folders exist: This will be done by the script.

## 4.3 How it does it

Your program must operate as a *dispatcher*: It is going to launch other, simpler programs. And it is going to do it by creating a new `compare` process for each image in the search folder, by first doing a `fork()`, followed by a call to an `execXY()` function by the child process.

The parent process will wait for all child processes to complete. As a child process completes, the dispatcher/parent is going to collect the result of the child’s search, and update its list of matches. When all child processes have terminated, the dispatcher will write the output file.

**Note:**

You must have the `fork/exec` behavior working in order to get execution points and the full “code quality” marks for this section. Just having the correct functionality performed by a single process misses the point of the assignment.

## 4.4 Extra credit (4 points)

If your program finds that a file with the same name as the one it is trying to create already exists in the output folder, then it should append a suffix to the name of the image, e.g. `clown [2].tga`, `clown [3].tga`

# 5 Part III: The Scripts

## 5.1 Location

As in the previous assignment, your scripts will go in the `Scripts` folder located at the root of the project folder, alongside the `ImageLibrary` and `Applications` folders. Your scripts should expect to be executed from the project’s root folder.

## 5.2 Build script

Your build script, named `build.sh` should build executable for all the applications.

## 5.3 Run script

Your run script will take three arguments:

- The path of the “source” directory to look for images in,
- the path to the search folder,
- The path to the output folder.

If the output folder doesn’t already exist, then your script should create it. If the folder already exists, then your script should delete its content.

For each image found in the source directory, the script will launch the `matchSearch` application, passing as arguments the path to the image, the path to the search folder, and the path to the output folder.

In a sense, what we get is a functionality that is a bit the reverse of what would be a really useful application (get as input a folder of new images, and then search the entire disk to see if we already have a copy of these images somewhere).

### 5.3.1 Version 1 of the script

This script, named `script03v1.sh` simply looks for files with the `.tga` extension in the source directory and calls `matchSearch` for each image.

### 5.3.2 Version 2 of the script

This script, named `script03v2.sh` looks *recursively* for files with the `.tga` extension in the source directory (so, it looks into subfolders of the search folders, and subfolders of these, etc.) and calls `matchSearch` for each image.

## 5.4 Extra credit: 4 points

If the search folder is in fact a subfolder of the source folder exclude each “source” image from its list of self-matches.

## 5.5 Advice

I recommend that you create a “sandbox” folder hierarchy with some hand-crafted text files here and there, and use this folder hierarchy to test your program. This should be general practice for all assignments in this course:

- It’s extremely easy to delete a file or folder by mistake in C, just by typing by mistake `fopen(..., "w")` instead `fopen(..., "r")` in your code. You don’t want to wipe out entire sections of your home folder<sup>4</sup>.
- If you have completely random text files in your folders and your script prints out that “23 occurrences of the target were found,” what are you going to do with that? Just assume that if the program didn’t crash and returned *something*, then it must be working, or go and check by hand all your files to verify that the result is correct? The first alternative is irresponsible programming and the second is a complete waste of time.
- You should test your script by running it with different starting points within your test folder hierarchy.

## 5.6 More extra credit

I will post more options for extra credit when questions start arriving on the forums.

# 6 Doxygen

## 6.1 Doxygen comments

You should provide Doxygen-style comments for all the function prototypes that you define.

---

<sup>4</sup>Particularly since I *know* that many of you are going to do all their development on their regular OS, mac OS or Windows, on their never backed-up admin account, needless to say. I don’t blame you for doing that. I do exactly the same (except for the admin account and lack of backup parts, that is).

## 6.2 Produce the documentation

Run Doxygen on the `ImageLibrary/include` folder to produce html documentation for the project. Name `Documentation` the folder containing the html documentation.

# 7 What to submit

## 7.1 Report

Along with the source code of the C program and shell script, you should submit a report in which you explain the algorithms of your exploration of the directory hierarchy (script) and match search (C program) and present any special data type or data structure that you needed to create for the project.

Identify any current limitation of your code: Can it crash? Under what circumstances? Do you make any assumptions in addition to the ones allowed in this assignment?

The report should be submitted as a `.pdf` file (Adobe Acrobat document). Not a plain text file, not a Word document, not a `.odt` file. A valid `.pdf` file.

## 7.2 Organization

Your project folder named `Prog03` should contain

- The `ImageLibrary` folder, with your additions;
- The `Applications` folder, with your `compare.c` and `matchSearch.c` source files;
- The `Scripts` folder;
- The `Documentation` folder containing the html documentation produced by Doxygen;
- The report.

but **not** the folder containing the images, nor the output folder, and definitely **no executable binaries**.

Zip the project folder (not `.rar`!) and submit on Brightspace before the due date. If you have problems on the way (broken computer, getting sick, etc.), you will get a more positive reception if you communicate early on than if you come up with your excuse on the last day, or even a couple of hours before the deadline.

## 7.3 Grading

- Execution: 40%
  - C code: 25%
    - \* `compare`: %
  - script code: 15%

- \* build script: 3%
  - \* run script V1: 4%
  - \* run script V2: 8%
- Quality of code: 25%
  - C code: 10%
  - script code: 5%
  - Doxygen comments: 5%
- Doxygen-produced documentation: 5%
- Report: 10%
- Folder organization (incl. files names as requested): 10%